



**BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONOURS**

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II: SESSION 2025/2026

Project Title:

Microservices-Based Disaster Relief Coordination Platform

GROUP 02

COURSE LECTURER:

DR. MOHAMAD NOR HASSAN

NAME	MATRIX NUMBER
MUHAMMAD ADHA BIN MUHAMMAD HAMKA	S76187
MUHAMMAD AIMAN MUKHLIS BIN KHAIRUM ANUAR	S76194
MUHAMMAD SYAKIR ZUFAYRI BIN SAARI	S74824

TABLE OF CONTENT

System Overview.....	3
Functional Requirements.....	4
Non-Functional Requirements.....	6
Architecture Style Explanation.....	8
Service Identification.....	9
Architecture Diagram.....	11
System Context Diagram.....	11
Service Architecture Diagram.....	13
Interaction Diagram.....	15
Deployment Diagram.....	21

System Overview

The Microservices-Based Disaster Relief Coordination Platform is designed to replace traditional monolithic systems, which are often too rigid and centralized to handle the chaos of emergencies. In a monolithic architecture, all features are tightly coupled into a single, massive program, meaning that a small change or a failure in one part of the system may affect the entire platform, making maintenance time-consuming and risky. These traditional designs often become slow or even crash when too many people try to access them at once, causing delays in rescue operations during critical situations. To overcome these limitations, the platform utilizes a microservices architecture that breaks the system down into a suite of independent services.

The system is designed to provide fast coordination in order to reduce the effects of the disaster in Malaysia, such as floods, earthquake, or pandemic, through the fulfillment of certain requirements. The first requirement is the ability of victims or volunteers to submit a report about the problem with the coordinates and pictures, which will then be sorted according to their priority and will be able to send a message to the closest person capable of providing help to the victim. Another aspect handled by this system is the logistics, where NGOs or organizations could report about their available food and medication supplies; meanwhile, the volunteer component will automatically assign individuals with their skills and location for a particular task.

From a strategic standpoint, the system has been designed in such a way as to provide the scalability and robustness required for a national disaster scenario. The fact that there are no centralized databases in the current approach enables server resources to be allocated precisely to the particular services that are receiving huge amounts of traffic, including incident reporting, without having to upgrade the entire system. The architecture is extremely resilient, as failure in any of the services does not necessarily mean that the whole system collapses; rather, all other services remain active.

Functional Requirements

1. User & Authentication Module

- a. FR-AUTH-01: The system shall validate user registration inputs (Victims, NGOs, Agencies, Volunteers) ensuring unique email addresses and password
- b. FR-AUTH-02: The API Gateway shall intercept all Incoming service requests and validate a JWT (JSON Web Token) before granting access to downstream microservices.

2. Reporting & Prioritization Module

- a. FR-REP-01: The system shall capture and store GPS coordinates (latitude/longitude) with high accuracy.
- b. FR-REP-02: The system shall automatically assign a "High" priority status to reports containing keywords such as "Trapped," "Medical Emergency," or "Flood Rising."
- c. FR-REP-03: The reporting module shall limit photo uploads to a maximum of 3 files per report, with a size limit of 5MB per image.

3. Resource Management Module

- a. FR-RSRC-01: Upon a resource request, the system shall query the inventory database and return a list of relief centers within a 50km radius that have the requested items in stock.
- b. FR-RSRC-02: The system shall decrement the "Available Stock" count in the database automatically once a delivery is marked as "Completed" by the driver.

4. Volunteer Management Module

- a. FR-VOL-01: System shall allow volunteers to register skills (medical, rescue, etc.) and automatically matches them to incidents based on location and skillset

- b. FR-VOL-02: Enables volunteers to accept or reject assigned tasks in real-time.

5. Communication Module

- a. FR-COM-01: Provides real-time messaging between victims, responders, NGOs, and government agencies.
- b. FR-COM-02 : The system shall log all communication history and message details to support official monitoring and post-disaster auditing

6. Notification Module

- a. FR-NOTIF-01: The system shall trigger alerts across multiple channels, including mobile push notifications (for app users), SMS, and email (for registered contacts)
- b. FR-NOTIF-02 : The system shall automatically trigger high-priority alerts for critical events, such as immediate evacuation orders, resource shortages, or high-risk incidents

Non-Functional Requirements

1. Scalability and Performance

- a. NFR-PERF-01: The system must handle sudden surges in user traffic without causing the platform to slow down or crash.
- b. NFR-PERF-02: The microservices architecture must allow for the precise allocation of server resources specifically to high-demand services (such as incident reporting) during a disaster, without requiring a full-system infrastructure upgrade.
- c. NFR-PERF-03: The decentralized data architecture must efficiently manage high volumes of concurrent read and write operations without creating data bottlenecks.

2. Reliability and Availability

- a. NFR-REL-01: The platform must maintain maximum system uptime, as availability during disaster scenarios directly impacts the ability to save lives.
- b. NFR-REL-02: The system must exhibit strong fault isolation; the failure or overwhelming of a single service must not cause the entire application to collapse, ensuring other services remain fully active.

3. Security

- a. NFR-SEC-01: The API Gateway must intercept all incoming requests and validate a JSON Web Token (JWT) before granting access to any downstream microservices.
- b. NFR-SEC-02: The system must securely handle and verify user credentials to guarantee that all registrations are distinct and secure.

4. Maintainability and Deployment

- a. NFR-MNT-01: Each microservice must be designed to focus on a distinct business task to allow for independent development, updates, and deployment without jeopardizing the stability of the entire platform.

Architecture Style Explanation

The Microservices-Based Disaster Relief Coordination Platform is proposed as a replacement for traditional, rigid monolithic systems, leveraging a microservices architecture. Centralized monolithic systems are highly susceptible to bottlenecks and potential crashes during disaster scenarios, especially when overwhelmed by a high volume of simultaneous user access. To mitigate these risks, the microservices approach is employed, which involves decomposing the application into a collection of independent and modular services.

- **High Scalability and Cost-Efficiency:**

- In emergencies, the surge in traffic typically targets specific functions, such as incident reporting. This microservices architecture enables precise allocation of server resources to the services experiencing heavy demand, avoiding the need for a full-system infrastructure upgrade.

- **Fault Isolation and Resilience:**

- Should a single service fail, whether from an error or overwhelming demand, the overall application will continue to operate. Given that system uptime is critical for a disaster relief platform and directly impacts the ability to save lives, this fault tolerance is essential.

- **Decentralized Data Management:**

- Each service operates with its own dedicated data store, eliminating reliance on a single central database. This architectural approach efficiently manages high volumes of concurrent reads and writes and avoids data bottlenecks across different modules.

- **Independent Deployment:**

- Focusing on a distinct business task, each microservice allows for independent development, updates, and deployment, which ensures the stability of the entire platform is not jeopardized.

Service Identification

- **API Gateway**
 - Serves as the unified access point for all incoming requests from various users, including Victims, Non-Governmental Organizations (NGOs), Government Agencies, and Volunteers.
 - It validates incoming requests by communicating with the Authentication Service to verify JSON Web Tokens (JWTs). Once validated, the requests are then routed to the appropriate downstream microservices.
- **Authentication Service**
 - Handles the registration of users and verifies the submitted data to guarantee that all credentials are distinct.
 - Verifies user credentials against the User Database and issues JSON Web Tokens (JWTs) to ensure secure platform-wide access.
- **Reporting Service**
 - Victims and volunteers can submit emergency reports through the system. These reports include high-accuracy GPS coordinates and support up to three photo attachments.
 - Incoming reports are automatically processed by the system, which assigns priority levels and categorization. Reports are automatically flagged as "High" risk if they contain critical terms, such as "Trapped" or "Medical Emergency."
 - The report data is stored separately within the Reporting Database.
- **Resource Management Service (Logistics)**
 - Facilitates registration and real-time inventory tracking for Non-Governmental Organizations (NGOs) and government institutions across various relief centers.
 - Queries the Inventory Database for available stock and processes resource requests.
 - The system interfaces with a Geographic Information System (GIS) to identify relief centers located within a 50km range. It also integrates with a Delivery Service to facilitate the automatic updating of stock counts upon successful completion of deliveries.

- **Volunteer Management Service (with Task Service)**

- Maintains and updates volunteer profiles, including detailed records of their specific skills (e.g., medical expertise, rescue training).
- In conjunction with the Task Service, this component is responsible for retrieving available tasks from the Task Database. It then performs automated matching, linking volunteers to specific incidents by considering key criteria such as the volunteer's location, availability, and skills.
- Manages the real-time process for volunteers to accept or reject their assigned tasks.

- **Communication Service**

- Enables real-time communication among victims, responders, and relief agencies.
- A dedicated Message DB stores all communication history, enabling official monitoring and post-disaster auditing.

- **Notification Service**

- Responsible for dispatching alerts to users through mobile push notifications, SMS, or email. This service is activated by other services, such as the Reporting or Communication services.
- Broadcasts essential, high-priority alerts regarding critical events, including immediate evacuations and severe shortages of vital resources.

Architecture Diagram

System Context Diagram

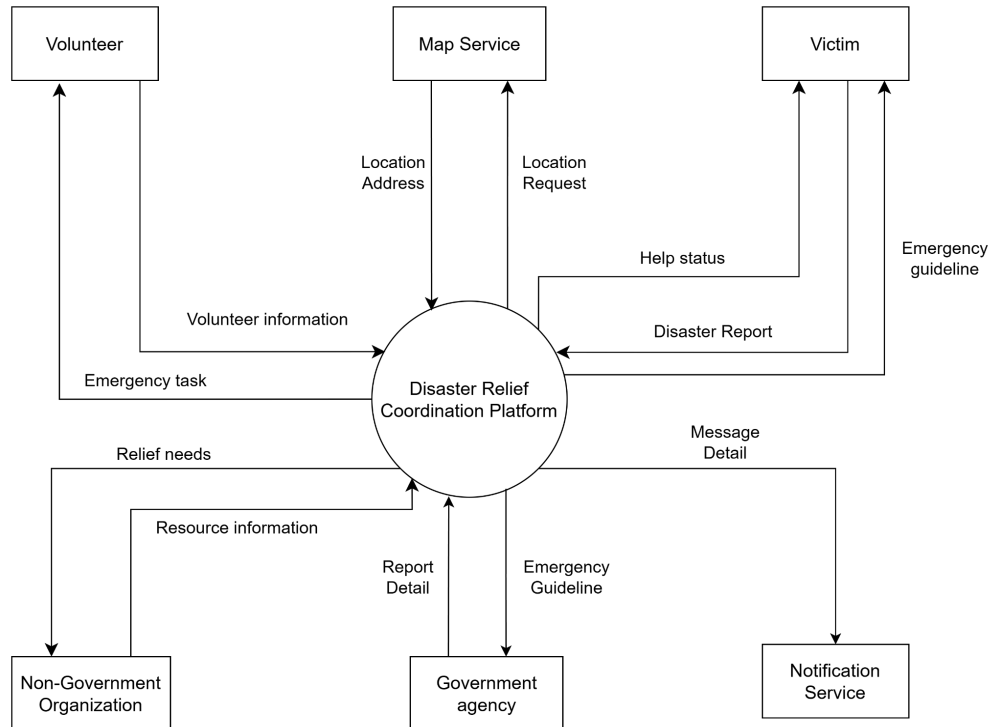


Figure 1.0 System Context Diagram

Description :-

1. Victim

- Input to Platform: Sends a Disaster Report to request help.
- Output from Platform: Receives Help status updates and Emergency guidelines for immediate safety instructions.

2. Volunteer

- Input to Platform: Provides Volunteer information (availability, skills, location).
- Output from Platform: Receives an Emergency task assigned by the system.

3. Government Agency (Official Government organization exp: Police, Ambulance and firefighter)

- Input to Platform: Provides Report Detail (official assessments or situational data).

- Output from Platform: Receives Emergency Guidelines for dissemination or implementation.

4. Non-Government Organization (NGO) Charity or private group

- Input to Platform: Provides Resource information (supplies, medical kits, food).
- Output from Platform: Receives Relief needs data to know where to deploy their resources.

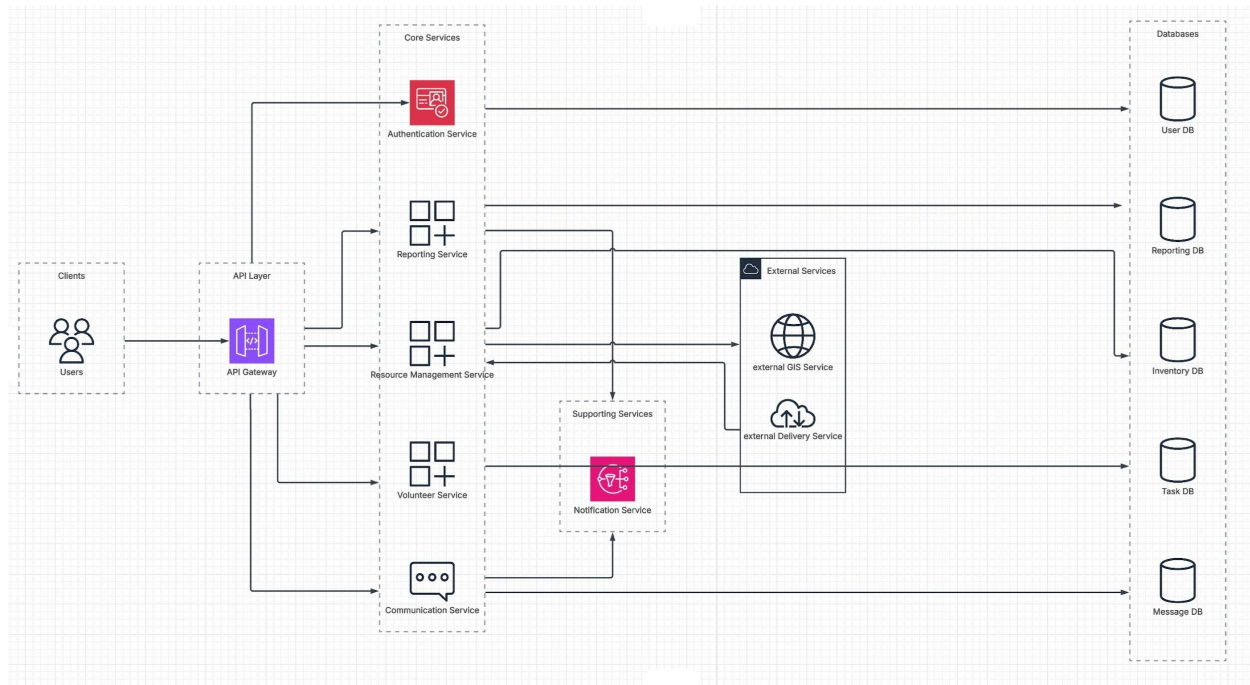
5. GIS Service AKA Map service

- Input to Platform: Sends Location Address data.
- Output from Platform: Receives a Location Request to geocode or map a specific report.

6. Notification Service (Communication like SMS and Push notification)

- Input to Platform: Sends Delivery status (confirming if a message reached its destination).
- Output from Platform: Receives Message Detail to be broadcasted to users.

Service Architecture Diagram



1. Users start by going through the API Gateway.
2. The API Gateway sends requests to five services: Authentication, Reporting, Resource Management, Volunteer, and Communication.
3. Each service has its own database.
4. The Authentication Service handles login and user access, and it uses the User DB.
5. The Reporting Service creates reports, stores data in the Reporting DB, and sends alerts through the Notification Service.
6. The Resource Management Service tracks supplies and resources, stores data in the Inventory DB, and connects to the GIS Service.
7. The Volunteer Service manages volunteer tasks and uses the Task DB.
8. The Communication Service handles messages, uses the Message DB, and also sends alerts through the Notification Service.
9. The Notification Service is shared by the Reporting and Communication services.
10. The Delivery Service connects to the Resource Management Service to help with delivery and logistics.

11. Overall, the diagram shows a system where each part has a clear job and works together to support disaster relief.

Interaction Diagram

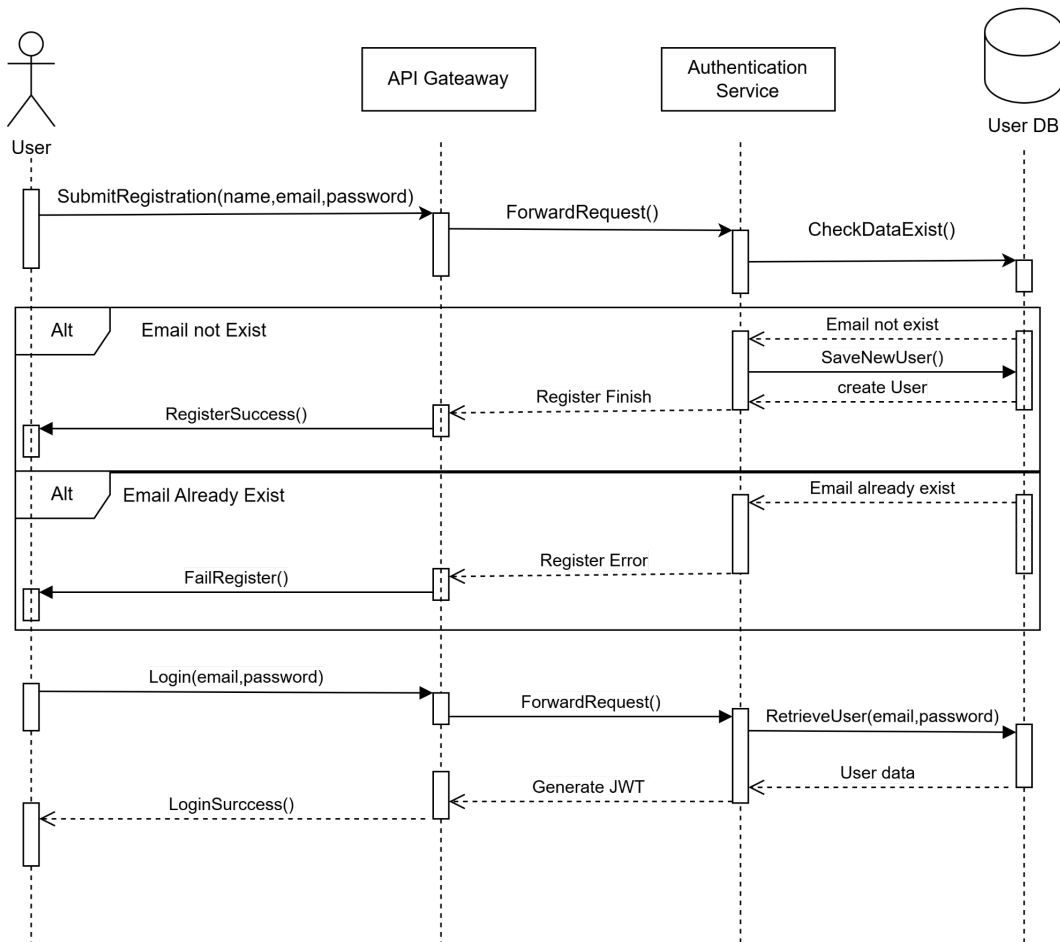


Figure Sequence Diagram Module User Registration & Login

Flow :-

1. The User submits their registration details, including their name, email, and password, to the API Gateway.
2. The API Gateway forwards this registration request to the Authentication Service for validation.
3. The Authentication Service checks with the User DB to see if the provided email address already exists in the system.
4. If the email does not exist:
 - a. The Authentication Service instructs the User DB to save the new user's information.

- b. Once the user is created, the Authentication Service notifies the API Gateway that the registration is finished.
 - c. The API Gateway confirms a successful registration to the User.
5. If the email already exists:
 - a. The User DB alerts the service that the email is a duplicate.
 - b. The Authentication Service sends a registration error message back to the API Gateway.
 - c. The API Gateway notifies the User that the registration has failed.
6. To access the system, the User submits their login credentials (email and password) to the API Gateway.
7. The API Gateway forwards the login request to the Authentication Service.
8. The Authentication Service attempts to retrieve the matching user record from the User DB.
9. Upon receiving the valid user data from the database, the Authentication Service generates a JWT (JSON Web Token).
10. The API Gateway receives the token and informs the User that the login was successful.

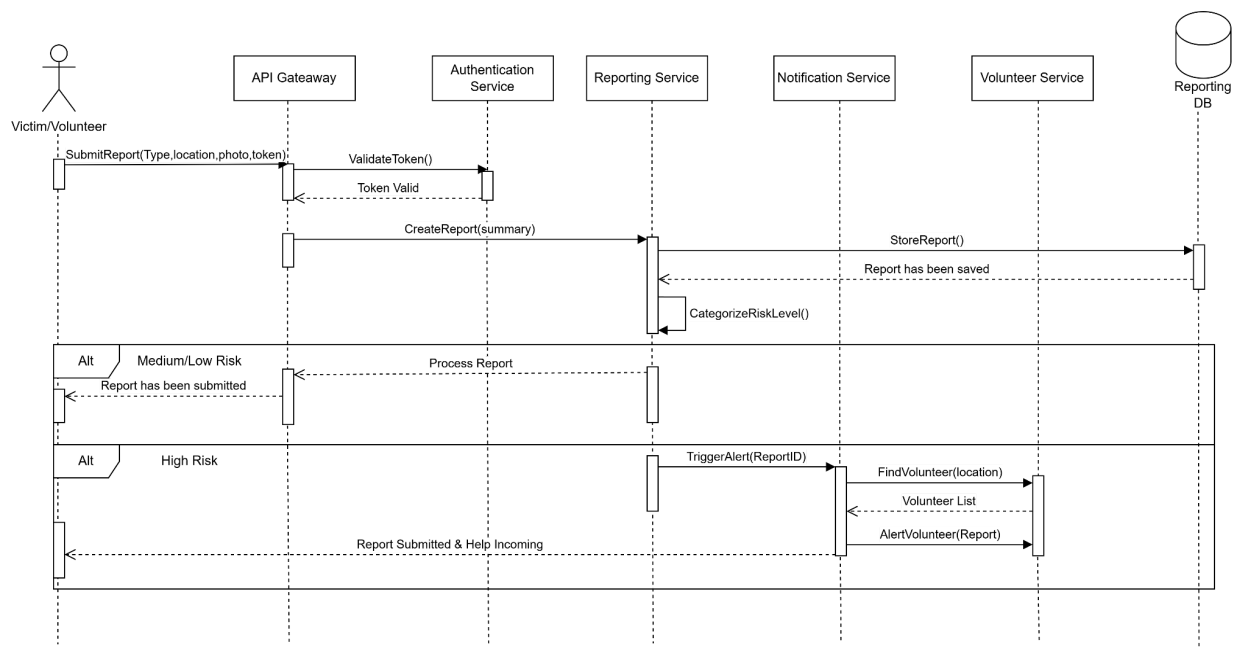


Figure Sequence Diagram Reporting Module

Flow :-

1. A Victim/Volunteer submits a report (type, location, photo, token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service to validate it.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway forwards the report request to the Reporting Service.
5. The Reporting Service creates the report summary.
6. The Reporting Service stores the report in the Reporting Database.
7. The database confirms that the report has been saved.
8. The Reporting Service categorizes the risk level of the report.
 - a. If the report is Medium or Low Risk, the system processes the report and sends a confirmation that the report has been submitted.
 - b. If the report is High Risk, the Reporting Service triggers an alert in the Notification Service.
 - i. The Notification Service requests nearby volunteers from the Volunteer Service based on the report location.
 - ii. The Volunteer Service returns a list of available volunteers.
 - iii. The Notification Service sends alerts to the selected volunteers.
 - iv. The system informs the user that the report has been submitted and help is on the way.

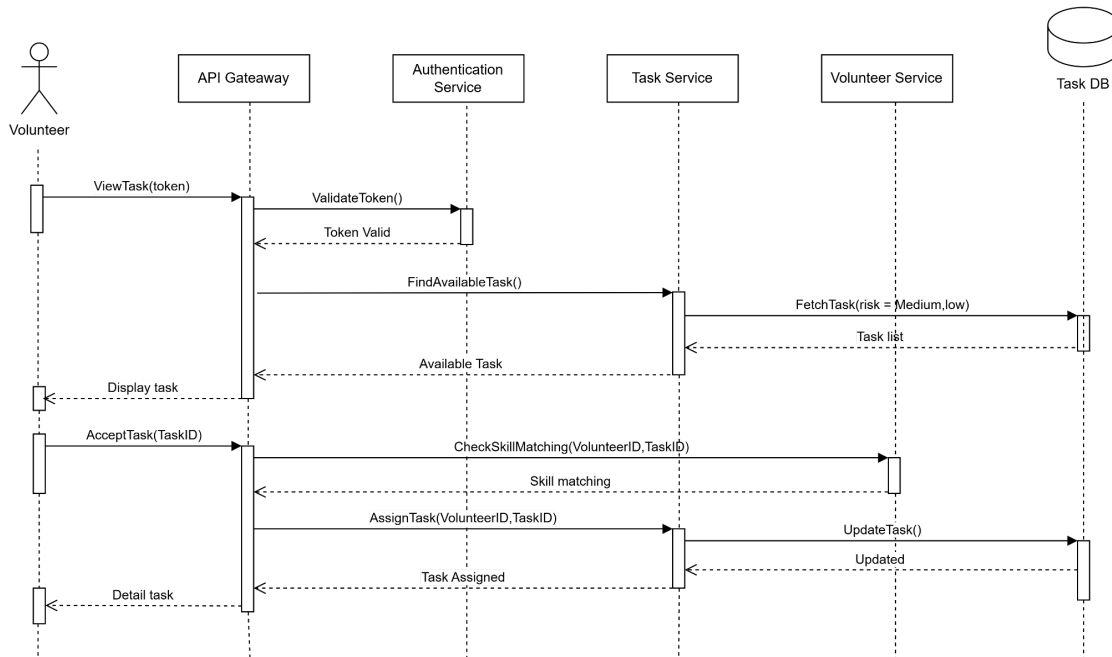


Figure Sequence Diagram Volunteer Management Module

Flow:-

1. A Volunteer sends a request to view tasks (with token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service for validation.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway requests available tasks from the Task Service.
5. The Task Service retrieves tasks from the Task Database with risk level (medium and low).
6. The Task Database returns the task list to the Task Service.
7. The Task Service sends the available tasks back to the API Gateway.
8. The API Gateway displays the task list to the Volunteer.
9. The Volunteer selects and sends a request to accept a task (TaskID) to the API Gateway.
10. The API Gateway requests the Volunteer Service to check skill matching using VolunteerID and TaskID.
11. The Volunteer Service verifies the skill compatibility and returns a “Skill matching” result.
12. The API Gateway sends a request to the Task Service to assign the task to the volunteer.
13. The Task Service updates the task in the Task Database.
14. The Task Database confirms that the task has been updated.

15. The Task Service returns a “Task Assigned” response to the API Gateway.
16. The API Gateway displays the task details to the Volunteer

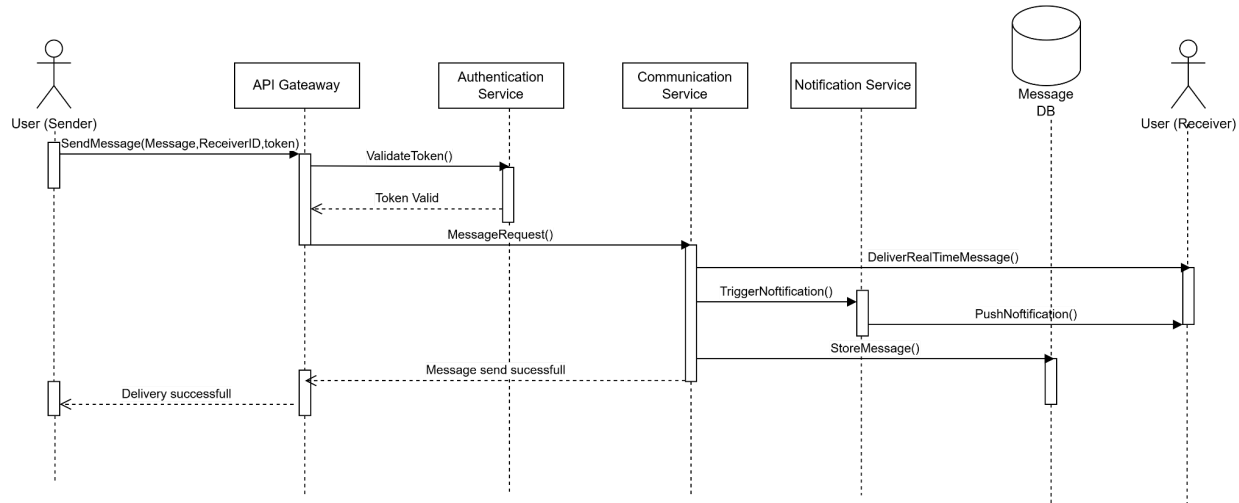


Figure Sequence Diagram Communication Module

Flow:-

1. A User (Sender) sends a message (message content, receiver ID, token) to the API Gateway.
2. The API Gateway sends the token to the Authentication Service for validation.
3. The Authentication Service verifies the token and returns a “Token Valid” response.
4. The API Gateway forwards the message request to the Communication Service.
5. The Communication Service processes the message request.
6. The Communication Service delivers the message in real time to the User (Receiver).
7. The Communication Service triggers the Notification Service to notify the receiver.
8. The Notification Service sends a push notification to the User (Receiver).
9. The Communication Service stores the message in the Message Database.
10. The Communication Service returns a “Message send successful” response to the API Gateway.
11. The API Gateway sends a “Delivery successful” confirmation back to the User (Sender).

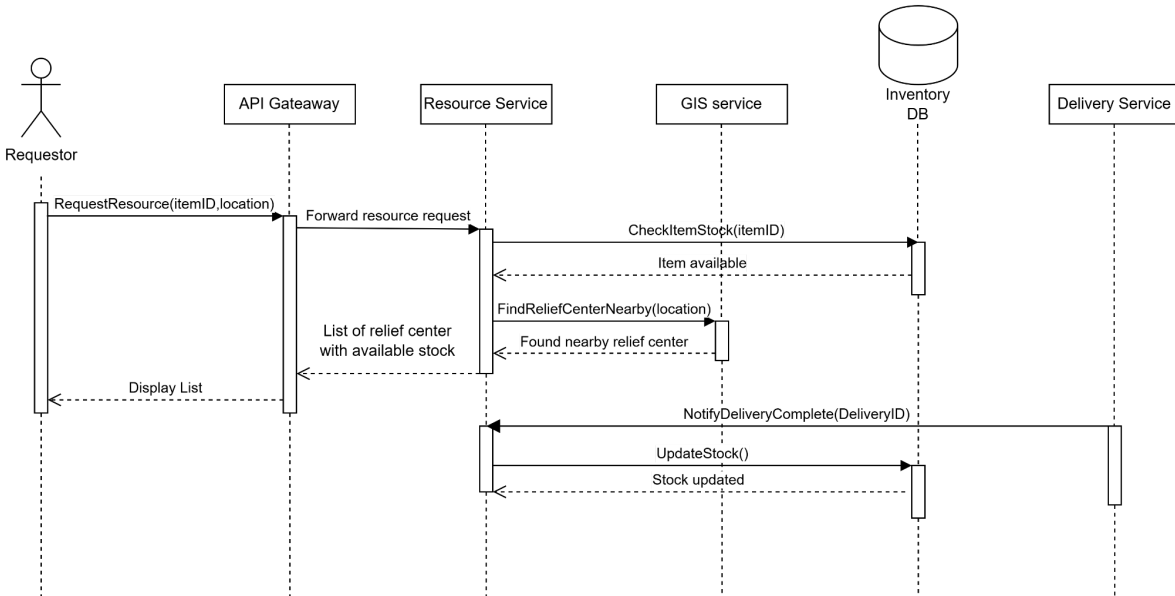


Figure Sequence Diagram Resource Management Module

Flow:-

1. The Requestor initiates a request for a specific resource by providing an item ID and their current location to the API Gateway.
2. The API Gateway acts as the entry point and forwards this request to the Resource Service for processing.
3. The Resource Service first queries the Inventory DB to verify if the requested item is currently in stock.
4. Once the Inventory DB confirms the item is available, the Resource Service contacts the GIS service to identify relief centers near the user's location.
5. The GIS service provides a list of nearby centers, which the Resource Service then filters based on where the stock is actually located.
6. This final list of available relief centers is sent back through the API Gateway and displayed to the Requestor.
7. At a later point, the Delivery Service triggers a notification to the Resource Service once a delivery has been successfully completed.
8. Upon receiving this completion notice, the Resource Service instructs the Inventory DB to update the stock levels to reflect the new delivery.
9. The Inventory DB confirms to the service that the records have been successfully updated.

Deployment Diagram

